

BASH

GUÍA ESENCIAL DEL SHELL



BearsLoveBeer

Índice

1. Introducción, historia, definiciones y contexto.....	1
1.1. Hitos cronológicos de Bash.....	1
1.1.1. (1977) El ancestro original: Bourne Shell (sh).....	1
1.1.2. (1988) El inicio del desarrollo.....	1
1.1.3. (1989) Lanzamiento oficial.....	2
1.1.4. (1990) Cambio de testigo.....	2
1.1.5. (2009) Migración de licencia (GPLv3).....	3
1.1.6. (2019 - 2025) Madurez y estabilidad (Versión 5.x).....	3
1.2. Conceptos fundamentales.....	3
2. Anatomía del comando y navegación.....	5
2.1. Comandos de orientación y rutas.....	5
2.2. Navegación entre directorios.....	5
2.3. Gestión de archivos y directorios.....	6
2.3.1. Manipulación de elementos.....	6
3. Comodines (Wildcards).....	7
4. Comandos avanzados y utilidades.....	8
4.1. Lectura, búsqueda y conteo.....	8
4.2. Redirecciones y pipes (tuberías).....	8
4.3. Variables y alias.....	9
5. Administración del Sistema (Permisos y Procesos).....	10
5.1. Gestión de permisos.....	10
5.1.1. El comando chmod: Modificando permisos.....	10
5.1.1.1. Modo Simbólico (Legible y sencillo).....	10
5.1.1.2. Modo Octal (Matemático y preciso).....	10
5.1.2. La Máscara de Permisos (umask).....	11
5.1.3. El Superusuario (root).....	11
5.2. Gestión de Procesos.....	11
6. Scripting con Bash y lógica.....	13
6.1. Estructura y Parámetros.....	13
6.2. Lógica de Control (Condicionales y Bucles).....	13
6.3. Funciones y Errores.....	15
7. Automatización con Cron.....	16
8. Bibliografía.....	17

1. Introducción, historia, definiciones y contexto.

Bash (Bourne Again SHell), el lenguaje de *scripting* más utilizado en el mundo para la línea de comandos y es el predeterminado en la inmensa mayoría de distribuciones Linux y macOS (hasta que este último cambió a Zsh, aunque sigue presente). La línea de comandos es crucial porque la **infraestructura de la nube, servidores y herramientas de despliegue** funcionan casi siempre sin interfaz gráfica. Es el lenguaje y la herramienta fundamental para la automatización, la administración de sistemas y el control de flujos de trabajo en entornos Unix.

1.1. Hitos cronológicos de Bash.

1.1.1. (1977) El ancestro original: Bourne Shell (sh).

Stephen Bourne introduce la **Bourne Shell (sh)** en Bell Labs, marcando un antes y un después en la historia de Unix. No fue una simple evolución, sino una revolución estructural que definió la interacción con el sistema operativo a través de tres pilares que Bash heredó intactos:

- **La herencia de ALGOL 68:** Fascinado por este lenguaje de programación, Bourne adoptó la convención de invertir palabras para cerrar bloques de código. Es la razón histórica por la que hoy en día cerramos un `if` con un `fi` o un `case` con un `esac`.
- **Un lenguaje completo, no un mero lanzador:** La shell anterior dependía de comandos externos y toscos (como `goto`). Bourne integró los condicionales y bucles de forma nativa, transformando la terminal en un lenguaje de programación *Turing completo* capaz de automatizar tareas lógicas complejas por sí mismo.
- **Cimientos de la gestión de datos:** Introdujo de forma robusta los **descriptores de archivo estándar (0, 1, 2)**, haciendo posible separar la salida limpia de los errores (`2> /dev/null`). También diseñó el comando `export`, permitiendo que los procesos hijos heredaran las variables del entorno del padre de manera limpia.

En conclusión: El diseño de 1977 convirtió la shell en una herramienta de desarrollo real. Estableció las bases arquitectónicas —y las peculiaridades sintácticas— que dominan la administración de sistemas actual.

1.1.2. (1988) El inicio del desarrollo.

En 1988, Brian Fox, bajo el mando de Richard Stallman y el **Proyecto GNU**, inició la creación de la *Bourne-Again SHell* (Bash). Este evento no fue solo un ejercicio técnico, sino una pieza fundamental de la misión de la Free Software Foundation para crear un ecosistema Unix completamente libre y abierto.

- **El juego de palabras y la identidad:** El nombre *Bourne-Again SHell* fue un guiño brillante al legado de Stephen Bourne. Al sonar como "*born-again*" (nacido de nuevo), no solo reconocía sus raíces en **sh**, sino que proclamaba una nueva era donde la shell pasaba a ser un software libre, sin las restricciones legales de los sistemas privativos de la época.
- **La filosofía de la compatibilidad:** El objetivo de Fox no fue reinventar la rueda, sino crear un **superconjunto compatible**. Bash se diseñó para que un administrador pudiera ejecutar sus scripts antiguos de **sh** sin modificaciones, mientras disfrutaba de nuevas capacidades avanzadas. Esto facilitó que la industria adoptara Bash casi instantáneamente.
- **El puente entre mundos:** Al integrar las mejores funcionalidades de otras shells populares de la época (como **csh** y **ksh**), Bash se convirtió en el punto de encuentro definitivo. Logró combinar la potencia de scripting de las shells más avanzadas con la accesibilidad y el modelo de licencias abiertas que exigía el movimiento GNU.

1.1.3. (1989) Lanzamiento oficial.

Se publica de forma oficial la primera versión beta de Bash.

1.1.4. (1990) Cambio de testigo.

Si Brian Fox dio vida a Bash, fue Chet Ramey quien le otorgó la estabilidad y el rigor técnico que lo convirtieron en un estándar industrial. Al asumir el cargo de mantenedor principal en 1990, Ramey transformó un proyecto ambicioso en una herramienta de precisión.

- **De la prototipación a la robustez:** Bajo su liderazgo, el desarrollo dejó de centrarse solo en añadir funcionalidades para enfocarse en la **corrección de errores y el cumplimiento estricto de estándares** (como POSIX). Esto eliminó las inconsistencias de comportamiento, logrando que un mismo script funcionara de forma idéntica en cualquier servidor del mundo.
- **Gestión del crecimiento orgánico:** Durante décadas, Ramey ha supervisado la integración de características complejas —desde la gestión avanzada de arrays asociativos hasta mejoras en la concurrencia— sin sacrificar la compatibilidad con versiones anteriores. Su capacidad para mantener el núcleo estable mientras Bash crecía en complejidad es lo que ha impedido que el proyecto se fragmentara.
- **Un estándar inamovible:** Gracias a su gestión constante, Bash pasó de ser "una alternativa libre" a ser la **lengua franca de la administración de sistemas**. La industria confía en Bash hoy en día precisamente por la solidez que Ramey ha cultivado, garantizando que sea una herramienta fiable tanto para tareas sencillas de escritorio como para la automatización de infraestructuras críticas a nivel global.

1.1.5. (2009) Migración de licencia (GPLv3).

El lanzamiento de Bash 4.0 en 2009 marcó un punto de inflexión estratégico no solo para el proyecto, sino para toda la industria tecnológica, al realizar la transición a la licencia **GPLv3**. Este cambio, impulsado por la Free Software Foundation, tuvo repercusiones profundas que alteraron el panorama de los sistemas operativos modernos.

- **El espíritu de la GPLv3:** Esta licencia introdujo cláusulas más estrictas contra la "tivoización" (la restricción de hardware que impide ejecutar software modificado) y reforzó la protección de las patentes. Para el proyecto Bash, significó blindar su filosofía de software libre, pero también creó una barrera infranqueable para empresas que buscaban un control cerrado sobre los componentes de su sistema.
- **El divorcio tecnológico con Apple:** Este hito es la causa directa de que, años más tarde, macOS decidiera congelar su versión de Bash en la antigua 3.2 (de 2007). Al no querer adoptar las condiciones de la licencia GPLv3, Apple optó por dejar de actualizar su intérprete predeterminado y, finalmente, sustituirlo por **Zsh** como *shell* por defecto en 2019, permitiéndoles mantener un control total sobre su entorno sin las ataduras de la nueva licencia.

1.1.6. (2019 - 2025) Madurez y estabilidad (Versión 5.x).

Se lanzan las versiones de la rama 5 (la más reciente estable, Bash 5.3, se publicó en julio de 2025), optimizando la gestión de memoria, la concurrencia en subshells y añadiendo soporte mejorado para arrays asociativos.

1.2. Conceptos fundamentales.

A continuación definiremos ciertos términos que creemos vitales tener claro desde un principio.

- **Shell:** Intérprete de comandos que traduce las órdenes de texto a acciones del sistema.
 - *Ejemplos de Shells:* **sh**, **zsh**, **CSH**, **PowerShell** y **Bash**.
- **Bash:** La *Shell* más popular y potente, surgida del proyecto GNU y evolución de *Born Shell*.
- **Terminal:** La interfaz o el programa gráfico que se abre para teclear comandos.
- **El Shebang (`#!/usr/bin/env bash`):** No es un comentario cualquiera al principio del programa; le indica al sistema operativo **qué intérprete debe usar** para ejecutar el archivo. Usar `/usr/bin/env` es una buena práctica porque busca la ubicación de Bash en el **PATH** del usuario, siendo mucho más portátil que escribir la ruta absoluta `/bin/bash`.
- **Variables de Entorno:** Son valores que viven en la memoria de tu Shell y que afectan cómo se comportan los programas (por ejemplo, el idioma o el editor por defecto).

- **El PATH:** Es la variable más importante. Es una lista de carpetas donde Bash busca los programas. Cuando escribes `git`, Bash revisa una por una las carpetas de tu `PATH` hasta encontrar el ejecutable de `git`. Si no está ahí, verás el famoso error `command not found`.
- **Kernel:** es el componente más profundo y fundamental de cualquier sistema operativo. Imagínalo como el **director de orquesta** que vive entre el hardware físico (tu procesador, memoria, disco duro) y el software que utilizas día a día (tu navegador, el editor de texto, el terminal). Sin el kernel, el software no tendría forma de comunicarse con el hardware de manera segura y eficiente). Cuando escribes `ls` en la terminal, la Shell le pide al Kernel que liste los archivos. Es el único que tiene permisos para acceder al hardware, y actúa como el "guardián" que gestiona la memoria y los procesos.

2. Anatomía del comando y navegación.

Un comando en Bash sigue la estructura general: **comando** **[opciones]** **[argumentos]**. Las opciones suelen indicarse con un guion (-).

2.1. Comandos de orientación y rutas.

Comando	Función Explicada	Ejemplo
<code>echo</code>	Imprime texto por pantalla (el "Hola Mundo").	<code>echo "hola bash"</code>
<code>pwd</code>	Muestra el directorio de trabajo actual (<i>Print Working Directory</i>).	<code>pwd</code>
<code>ls</code>	Lista el contenido del directorio actual.	<code>ls</code>
<code>ls -l</code>	Lista el contenido en formato largo (muestra permisos, propietario, fecha).	<code>ls -l</code>
<code>ls -a</code>	Incluye la visualización de archivos ocultos .	<code>ls -a</code>
<code>man</code>	Muestra la documentación o manual de un comando (se sale pulsando <code>Q</code>).	<code>man ls</code>
<code>clear</code>	Limpia la pantalla de la terminal.	<code>clear</code>
<code>whoami</code>	Muestra el nombre de usuario del sistema.	<code>whoami</code>
<code>uname -a</code>	Muestra información detallada sobre el kernel y el sistema.	<code>uname -a</code>

2.2. Navegación entre directorios.

El comando `cd` (*Change Directory*) permite cambiar el directorio de trabajo.

Comando	Explicación	Ejemplo
Mover hacia adelante	Entrar en un subdirectorio.	<code>cd curso</code>
Mover hacia atrás	Subir un nivel en la jerarquía.	<code>cd ..</code>

Directorio Home	Ir directamente a la carpeta raíz del usuario.	<code>cd ~</code>
Rutas absolutas	Ubicación completa desde la raíz, ignora la posición actual.	<code>cd /users/dadomher/workspac e</code>
Rutas relativas	Referencia al directorio actual.	<code>cd cursos/config</code>

2.3. Gestión de archivos y directorios.

Esta sección se centra en la manipulación directa de archivos y carpetas.

2.3.1. Manipulación de elementos.

Comando	Función	Ejemplo
<code>mkdir</code>	Crea un directorio o carpeta.	<code>mkdir carpeta_nueva</code>
<code>rm</code>	Elimina archivos de forma definitiva (no van a la papelera).	<code>rm archivo.txt</code>
<code>rm -r</code>	Eliminación recursiva (necesario para borrar directorios con contenido).	<code>rm -r curso</code>
<code>rm -ri</code>	Eliminación recursiva interactiva (solicita confirmación antes de borrar).	<code>rm -ri carpeta</code>
<code>rm -rf</code>	Eliminación recursiva forzada (muy peligroso, borrado definitivo sin confirmación).	<code>rm -rf /directorio</code>
<code>cp</code>	Copia archivos.	<code>cp licencia licencia_copia</code>
<code>cp -a/cp -r</code>	Copia directorios y su contenido de forma recursiva.	<code>cp -r course curso_backup</code>
<code>mv</code>	Mueve o renombra un archivo/directorio.	<code>mv licencia curs (Mover)</code>
<code>mv</code>	Renombra un archivo si el destino está en el mismo directorio con nombre diferente.	<code>mv README.md LM.md (Renombrar)</code>

3. Comodines (*Wildcards*).

Los comodines se utilizan para referenciar múltiples archivos de forma rápida y funcionan en conjunto con comandos como `ls`, `rm` o `mv`.

- *** (Asterisco):** Representa **cero o más caracteres**.
 - *Ejemplo:* Listar todos los archivos con extensión .md: `ls *.md`
 - *Ejemplo:* Borrar todos los archivos que empiecen por **03**: `rm 03*`
- **? (Interrogación):** Representa un **número exacto de caracteres** (uno por signo).
 - *Ejemplo:* Borrar archivos con solo una letra en el nombre y extensión .txt: `rm ?.txt`

4. Comandos avanzados y utilidades.

4.1. Lectura, búsqueda y conteo.

A continuación, los comandos esenciales para la manipulación, búsqueda y conteo de datos en el sistema.

Comando	Función	Ejemplo
<code>cat</code>	Muestra el contenido completo de un archivo (útil para archivos pequeños).	<code>cat licencia</code>
<code>less</code>	Muestra el contenido de forma paginada (ideal para archivos largos; se sale con Q).	<code>less licencia</code>
<code>head</code>	Muestra las 10 primeras líneas por defecto.	<code>head -n 20 licencia</code> (Muestra las primeras 20 líneas)
<code>tail</code>	Muestra las 10 últimas líneas por defecto.	<code>tail -f file.log</code> (Sigue el archivo en tiempo real)
<code>grep</code>	Busca patrones o cadenas de texto dentro de archivos o en la salida de comandos.	<code>grep "object" licencia</code>
<code>grep -i</code>	Ignora mayúsculas y minúsculas durante la búsqueda.	<code>grep -i "object" licencia</code>
<code>grep -r</code>	Búsqueda recursiva en directorios y subdirectorios.	<code>grep -r "bash" curs</code>
<code>wc</code>	Cuenta líneas, palabras y bytes.	<code>wc -l licencia</code> (solo líneas)
<code>find</code>	Busca archivos por nombre o criterio en el sistema de archivos.	<code>find . -name "README.md"</code>

4.2. Redirecciones y pipes (tuberías).

Estos símbolos permiten controlar el flujo de entrada y salida de los comandos.

- **> (Redirección):** Envía la salida a un archivo, **sobrescribiendo** el contenido.
 - *Ejemplo:* `echo "hola" > hello.txt.`
- **>> (Adición):** Envía la salida a un archivo, **añadiendo** al final.
 - *Ejemplo:* `echo "dadomher" >> hello.txt.`

- **< (Entrada):** Toma la entrada de un archivo para un comando.
 - *Ejemplo:* `sort < hello.txt` (ordena el contenido del archivo).
- **| (Pipe/Tubería):** Concatena la salida de un comando como entrada para el siguiente.
 - *Ejemplo:* `cat licencia | grep "object" | wc -w` (Muestra el conteo de palabras de las líneas que contienen "object").

4.3. Variables y alias.

- **Variables locales:** Almacenan datos en la sesión actual de la terminal.
 - *Ejemplo:* `name=bruce`, seguido de `echo $name`.
- **Variables globales (Entorno):** Accesibles desde nuevas sesiones, creadas con `export` y persistidas en el *script* de configuración de Bash.
 - *Ejemplo:* `export NAME="Daniel Domínguez"`.
- **Alias (Atajos):** Crean una abreviación temporal para un comando largo.
 - *Ejemplo de Creación:* `alias ll='ls -LH'`.
 - *Ejemplo de Uso:* `ll` (ejecuta `ls -LH`).
 - *Ejemplo de Eliminación:* `unalias ll`.

5. Administración del Sistema (Permisos y Procesos).

5.1. Gestión de permisos.

En Linux/Unix, cada archivo tiene tres tipos de permisos básicos: **Lectura (R=4)**, **Escritura (W=2)** y **Ejecución (X=1)**. Aplicables a tres niveles: **Usuario** (tú), **Grupo**, **Otros** y **A todos**.

5.1.1. El comando chmod: Modificando permisos.

`chmod` (*change mode*) permite cambiar quién puede hacer qué con un archivo.

5.1.1.1. Modo Simbólico (Legible y sencillo).

Utiliza letras para definir quién, qué acción y qué estado aplicar:

- **Quién:** `u` (usuario), `g` (grupo), `o` (otros), `a` (todos).
- **Acción:** `+` (añadir), `-` (quitar), `=` (asignar exactamente).
- **Permiso:** `r` (lectura), `w` (escritura), `x` (ejecución).

Ejemplo: `chmod u+x script.sh` (añade permiso de ejecución al usuario).

Ejemplo: `chmod g-w archivo.txt` (quita permiso de escritura al grupo).

5.1.1.2. Modo Octal (Matemático y preciso).

Utiliza un número de 3 dígitos (del 0 al 7) donde cada número es la suma de: **Lectura (4) + Escritura (2) + Ejecución (1)**.

- **7** (4+2+1): Lectura, escritura y ejecución.
- **6** (4+2): Lectura y escritura.
- **5** (4+1): Lectura y ejecución.
- **4**: Solo lectura.

Ejemplo: `chmod 755 script.sh`

- **7 (Usuario):** `rwx`
- **5 (Grupo):** `r-x`
- **5 (Otros):** `r-x`

5.1.2. La Máscara de Permisos (umask).

La **umask** es una "máscara" que define qué permisos **se deben quitar** automáticamente al crear un archivo nuevo. Es una medida de seguridad para que los archivos no se creen con permisos totales por defecto.

- Si la máscara es **022**:
 - Los archivos nuevos restan 2 (escritura) al grupo y otros.
 - Resultado: El archivo se crea con **644** (lectura/escritura para ti, solo lectura para el resto).
- Se consulta escribiendo **umask** en la terminal.

5.1.3. El Superusuario (root).

El usuario **root** (o superusuario) es el administrador del sistema. Tiene privilegios totales y puede saltarse todas las restricciones de permisos del Kernel.

- **sudo (SuperUser DO)**: Es el comando que permite a un usuario normal ejecutar tareas administrativas de forma temporal y controlada.
- **Precaución**: Como el Kernel no limita al superusuario, un error con **sudo** (ej. borrar un sistema de archivos) no tiene marcha atrás. Úsalo solo cuando sea estrictamente necesario.

5.2. Gestión de Procesos.

Control y monitorización de procesos: Comandos fundamentales para la gestión del ciclo de vida de las tareas en el sistema.

Comando	Función	Ejemplo
ps aux	Lista todos los procesos en ejecución.	ps aux
top	Muestra los procesos de modo interactivo y en tiempo real.	top
jobs	Muestra los trabajos que se ejecutan en segundo plano .	jobs
sleep	Detiene la ejecución por N segundos, útil para crear procesos en segundo plano.	sleep 100
Control + Z	Suspende un proceso que se ejecuta en primer plano.	

<code>fg</code>	Trae un proceso que está en segundo plano al primer plano (<i>Foreground</i>).	<code>fg 1</code>
<code>bg</code>	Reanuda un proceso detenido en segundo plano (<i>Background</i>).	<code>bg 1</code>
<code>kill [PID]</code>	Elimina un proceso usando su PID.	<code>kill 1234</code>
<code>kill -9 [PID]</code>	Fuerza la detención de un proceso.	<code>kill -9 1234</code>
<code>history</code>	Muestra el historial de comandos lanzados.	<code>history</code>
<code>!!</code>	Repite el último comando ejecutado.	<code>!!</code>
<code>!10</code>	Repite el comando cuyo ID en el historial es 10.	<code>!date</code>

6. Scripting con Bash y lógica.

Un **script** es un archivo de texto con extensión `.sh` que contiene comandos que la *Shell* ejecuta secuencialmente.

6.1. Estructura y Parámetros.

Guía de construcción: Estructura básica y manejo de variables de script.

- **Shebang:** Indica el intérprete.
 - *Ejemplo:* `#!/bin/bash`.
- **Ejecución:** Requiere permisos de ejecución (e.g., `chmod +x script.sh`).
 - *Ejemplo:* `./script.sh`.
- **Comentarios:** Se definen con la almohadilla.
 - *Ejemplo:* `# Mi primer script`.
- **Variables y Aritmética:**
 - *Definición:* `name=brace`.
 - *Interpolación de comandos:* `echo "Tu directorio es $(PWD)"`.
 - *Aritmética moderna:* `SUMA=$(( A + B ))` (Suma, resta, multiplicación, división, módulo).
- **Lectura de Usuario (`read`):**
 - *Ejemplo Básico:* `read name`.
 - *Ejemplo con Prompt:* `read -p "¿Cuál es tu edad?" H`.
 - *Ejemplo Oculto:* `read -s password` (para contraseñas).
- **Argumentos de Script (Parámetros):**
 - `$0`: El nombre del script.
 - `$1`, `$2`: Los parámetros pasados.
 - `$#`: El número total de parámetros.
 - *Ejemplo de Ejecución con Parámetros:* `./paramscript brace dadomher`.

6.2. Lógica de Control (Condicionales y Bucles).

Control de flujo: Tomando decisiones y automatizando tareas con condicionales y bucles.

- **Condicionales (`if`, `elif`, `else`, `fi`):**

Ejemplo de Estructura:

```
if [ $num -ge 10 ]; then
    echo "Mayor o igual que 10"
elif [ $num -eq 0 ]; then
    echo "Cero"
else
```

```
echo "Condición por defecto"
fi
```

- (Nota: Los espacios son estrictos en Bash).
- *Operadores de Comparación (Numéricos)*: `-eq` (igual), `-ne` (distinto), `-gt` (mayor que).
- *Operadores de Comparación (Cadenas)*: `-z` (vacía), `-n` (no vacía).
- *Comprobación de Archivos*: `-e` (archivo existe).
- *Operadores Lógicos*: `&&` (AND) y `||` (OR).
- **Condicional `case`**: Comprueba un valor contra múltiples patrones.
 - *Ejemplo*:

```
case $option
in 1)
    echo "Elegiste el 1";
;
*) echo "Opción no válida";
esac.
```

- **Bucle `for`**: Itera sobre una lista definida de valores o archivos.
 - *Ejemplo (Lista de valores)*:

```
for i in 1 2 3 4 5;
do echo "Número $i";
done.
```

- *Ejemplo (Archivos)*:

```
for name in *.sh;
do echo "Archivo $name";
done.
```

- **Bucle `while`**: Repite el código **mientras** la condición sea verdadera.
 - *Ejemplo*:

```
while [ $count -le 5 ];
do echo $count;
count=$((count+1));
done.
```

- **Bucle `until`**: Repite el código **hasta que** la condición sea verdadera.
- **Control de Bucles**:
 - `continue`: Salta a la siguiente iteración.
 - `break`: Detiene el bucle por completo.

6.3. Funciones y Errores.

Aprende lo básico para encapsular lógica mediante funciones y a gestionar el flujo de ejecución ante cualquier fallo inesperado.

- **Funciones:** Permiten reutilizar código.
 - *Definición:* `function my_function() { echo "Hola"; }`.
 - *Llamada:* `my_function`.
 - *Retorno:* Se devuelve un valor de salida con `return`.
- **Manejo de Errores:** Se accede al código de salida del último comando con `$??`.

Cero (`0`) indica éxito; distinto de cero indica error.

- *Comprobación de error:*

```
if [ $? -ne 0 ]; then echo "Error al copiar el archivo";  
fi.
```

- *Control de errores en una línea (Doble Pipe):*

```
cp file.txt curso || echo "Error"
```

(El mensaje se ejecuta solo si `cp` falla).

7. Automatización con Cron.

Cron es un *demonio* (proceso del sistema) que ejecuta **tareas programadas** automáticamente en segundo plano.

- **Comandos de `crontab`:**

`crontab -l`: Lista las tareas configuradas.

`crontab -e`: Abre el editor para configurar la tabla.

`crontab -r`: Elimina toda la tabla de tareas.

- **Sintaxis de tarea Cron:** Se define con cinco campos de tiempo seguidos del comando (usando rutas absolutas). | Campo | Rango | | :--- | :--- | | 1. Minuto | 0-59 | | 2. Hora | 0-23 | | 3. Día del Mes | 1-31 | | 4. Mes | 1-12 | | 5. Día de la Semana | 0-6 (0 = Domingo) |

- **Ejemplo de Tarea Cron:**

- *Ejecutar un script cada minuto, redirigiendo la salida a un log:*
`* * * * * /ruta/al/script.sh >> /ruta/al/cron.log`
- *Ejecutar a las 2:30 AM todos los días:*
`30 2 * * * /comando/a/ejecutar`
- *Ejecutar cada 5 minutos:*
`* /5 * * * * /comando/a/ejecutar`
(El asterisco seguido de barra indica recurrencia).

8. Bibliografía.

GNO corp. "Bash." *GNU*, 22 September 2020, <https://www.gnu.org/software/bash/>.

Accessed 27 May 2026.

GNU org. "Bash Features." *GNU*, 2025,

<https://www.gnu.org/software/bash/manual/bash.html>. Accessed 27 05 2026.

Moure, Brais. "Guía rápida de Bash/Shell, terminal y línea de comandos." *mouredev pro by*

Brais Moure, 2024,

https://campus.mouredev.pro/products/digital_downloads/guia-bash. Accessed 27

May 2026.

The Open Group. "Shell Command Language (POSIX.1-2024)." *The Open Group*, 2024,

https://pubs.opengroup.org/onlinepubs/9799919799/utilities/V3_chap02.html.

Accessed 27 05 2026.

S. R., Bourne. "The Bell System Technical Journal." *The UNIX Shell*, 1978,

<https://archive.org/details/bstj57-6-1971>. Accessed 27 05 2026.